Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !’ DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !’ MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !’ FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !’ QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: SCRUM-25 — Dashboard 'The Power Grid'

1. CONTEXTO
Se requiere un dashboard interactivo de temática cyberpunk/mission control que simule un centro de control de integración Atlassian. El requerimiento especifica explícitamente que debe ejecutarse localmente abriendo un archivo `index.html` en el navegador (protocolo `file:///`). El stack está fijado por el cliente: HTML5 semántico, CSS3 puro y Vanilla JavaScript ES6+. No hay backend, no hay servidor, no hay build tools.

2. DECISIÓN ARQUITECTÓNICA
Arquitectura elegida: Single-File Application (SFA) — Monolito HTML Autocontenido.

Toda la lógica JavaScript se empaquetará en un único archivo `main.js` referenciado con `<script src='main.js'>` (SIN `type='module'`). El CSS se externaliza en `style.css` por mantenibilidad. La aplicación completa se compone de exactamente 2 archivos funcionales + 1 punto de entrada HTML.

EVALUACIÓN DE ENTORNO (OBLIGATORIA):
- Entorno detectado: **LOCAL** (`file:///`)
- Protocolo: Sin servidor HTTP
- Consecuencia directa: **PROHIBIDO** usar `<script type='module'>`, `import/export` nativos de ES6, o dividir la lógica JS en múltiples módulos con imports. Todo el JavaScript debe vivir en un único archivo plano (`main.js`) o inline en el HTML. Se elige `main.js` externo por legibilidad, pero sin módulos ES6.

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS
- **¿Por qué NO un bundler (Webpack/Vite)?** YAGNI. El requerimiento no menciona un pipeline de build. Agregar un bundler introduce una dependencia de Node.js, npm y configuración que el usuario final no puede ejecutar con doble-clic. Costo: alto. Beneficio para este MVP: cero.
- **¿Por qué NO múltiples archivos JS con módulos?** El entorno `file:///` bloquea CORS en módulos ES6 nativos en la mayoría de navegadores (Chrome, Firefox). Dividir en módulos rompería la aplicación sin un servidor HTTP. Decisión: un único `main.js` con namespacing manual mediante objetos literales (`const PowerGrid = { state: {}, render: {}, kpi: {} }`).
- **¿Por qué NO un framework (React/Vue)?** El requerimiento lo prohíbe explícitamente ('sin frameworks de UI externos'). Además, para una SPA de visualización estática con Canvas, Vanilla JS es la herramienta correcta.
- **¿Por qué NO separar Canvas en workers?** YAGNI. El requerimiento pide ~60 FPS con `requestAnimationFrame`. Para 2 canvas (partículas + conexiones) y ~50-100 partículas, el hilo principal es más que suficiente. OffscreenCanvas añadiría complejidad sin beneficio medible.
- **¿Por qué NO una base de datos o backend?** El estado es 100% efímero (5 booleanos + 1 flag overdrive). No hay persistencia requerida. `window` global es suficiente.

4. ESTRUCTURA DE ESTADO Y CONFIGURACIÓN
- **`globalState`**: Objeto plano con `modules: { gitlab: bool, gitlabCI: bool, xray: bool, datadog: bool, rovo: bool }` y `overdrive: bool`.
- **`KPI_CONFIG`**: Objeto de configuración con valores base, boosts por módulo, min/max y referencias DOM. Centraliza toda la lógica de negocio.
- **Motor de render**: `requestAnimationFrame` único que orquesta: (1) limpiar canvas, (2) dibujar partículas, (3) dibujar conexiones + pulsos, (4) actualizar KPIs animados.

5. RIESGOS ASUMIDOS POR SIMPLIFICAR
- **Riesgo 1 (BAJO):** Al tener todo en `main.js`, el archivo será largo (~600-900 líneas). Mitigación: secciones claramente comentadas con separadores (`// === SECTION: CANVAS ENGINE ===`).
- **Riesgo 2 (BAJO):** Sin módulos, no hay tree-shaking. Irrelevante para un archivo local sin red.
- **Riesgo 3 (MEDIO):** Si en el futuro se requiere persistencia o multi-usuario, la arquitectura deberá refactorizarse. Aceptado conscientemente: YAGNI dicta no diseñar para requisitos hipotéticos.
- **Riesgo 4 (BAJO):** Compatibilidad de Canvas API: todos los navegadores modernos (Chrome 90+, Firefox 88+, Edge 90+) soportan la API 2D completa requerida.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.js | Estado: APROBADO | Iteraciones: 1
- style.css | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 39,721
Tokens de salida (output): 18,472
Total tokens: 58,193
Horas-hombre estimadas: ~6.4 horas
Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“3,097 input | !‘1,475 output | 1 llamadas
MICROPLANIFICACION-DISEÑO: !“7,752 input | !‘4,500 output | 3 llamadas
FRONTEND-DESARROLLO: !“7,981 input | !‘12,386 output | 3 llamadas
QA-VALIDACION: !“20,891 input | !‘111 output | 3 llamadas